Pętle w programowaniu



Umożliwia cykliczne wykonywanie ciągu instrukcji określoną liczbę razy, do momentu zajścia pewnych warunków, dla każdego elementu kolekcji lub w nieskończoność.

```
while warunek:
    # kod
```

Instrukcja **while** wykonuje pewien fragment kodu tak długo dopóki warunek w niej określony jest spełniony. warunek musi być interpretowany w kategoriach True i False kod który ma zostać wykonany po spełnieniu warunku (True) musi znajdować się w następnej linijce i być poprzedzony wcięciem może być wiele instrukcji podlegających wielokrotnemu wykonaniu, jednak wszystkie one muszą być poprzedzone takim samym wcięciem.

Pętle w programowaniu



Zmienna do obsługi pętli

```
i = 0
```

Warunek pętli

```
while(i < 10):
    print(i)
    i += 1
```

Ciało pętli (jeśli warunek pętli zwróci True, to kod wewnątrz ciała się wykona)

Aby zdefiniować pętlę, należy użyć słowa kluczowego while.
Aby pętla się wykonała warunek pętli musi zwracać True.

Stan zmiennej liczba w każdym obiegu pętli:

i = 0
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9



i = 0

Wypisz 0
Zwiększ o 1
i = 1

Wypisz 1
Zwiększ o 1
i = 2

Wypisz 2
Zwiększ o 1
i = 3

Wypisz 3
Zwiększ o 1
i = 4

Wypisz 4
Zwiększ o 1
i = 5

Wypisz 5
Zwiększ o 1
i = 6

Wypisz 6
Zwiększ o 1
i = 7

Wypisz 7
Zwiększ o 1
i = 8

Wypisz 8
Zwiększ o 1
i = 9

Pętle w programowaniu



```
10
9
8
7
6
5
4
3
2
1
0
```

```
i = 10

while(i >= 0):

    print(i)
    i -= 1
```

Stan zmiennej liczba w każdym obiegu pętli:

i = 10
i = 9
i = 8
i = 7
i = 6
i = 5
i = 4
i = 3
i = 2
i = 1
i = 0



Wypisz 10 Zmniejsz o 1 i = 9
Wypisz 9 Zmniejsz o 1 i = 8
Wypisz 8 Zmniejsz o 1 i = 7
Wypisz 7 Zmniejsz o 1 i = 6
Wypisz 6 Zmniejsz o 1 i = 5
Wypisz 5 Zmniejsz o 1 i = 4
Wypisz 4 Zmniejsz o 1 i = 3
Wypisz 3 Zmniejsz o 1 i = 2
Wypisz 2 Zmniejsz o 1 i = 1
...

i = 10 ...

Pętla while przykłady



```
counter = 0
while counter < 4:
    print(counter)
    counter += 1
```

```
0
1
2
3
```

```
counter = 4
while counter > 0:
    print(counter)
    counter -= 1
```

```
4
3
2
1
```

Przestrzeń lokalna i globalna



Przestrzeń globalna

Przestrzeń lokalna

Przestrzeń globalna

```
counter = 0
while counter < 4:
    print(counter)
    counter += 1
print("Przestrzen globalna")
```

```
4
3
2
1
Przestrzen globalna
```

```
    print(counter)
        ^
IndentationError: expected an indented block
```

```
counter = 4
while counter > 0:
print(counter)
counter -= 1
```

Pętla nieskończona while



```
while True:
    print("pomocy!")
```

```
pomocy!
pomocy!
pomocy!
pomocy!
pomocy!
Process finished with exit code -1
```

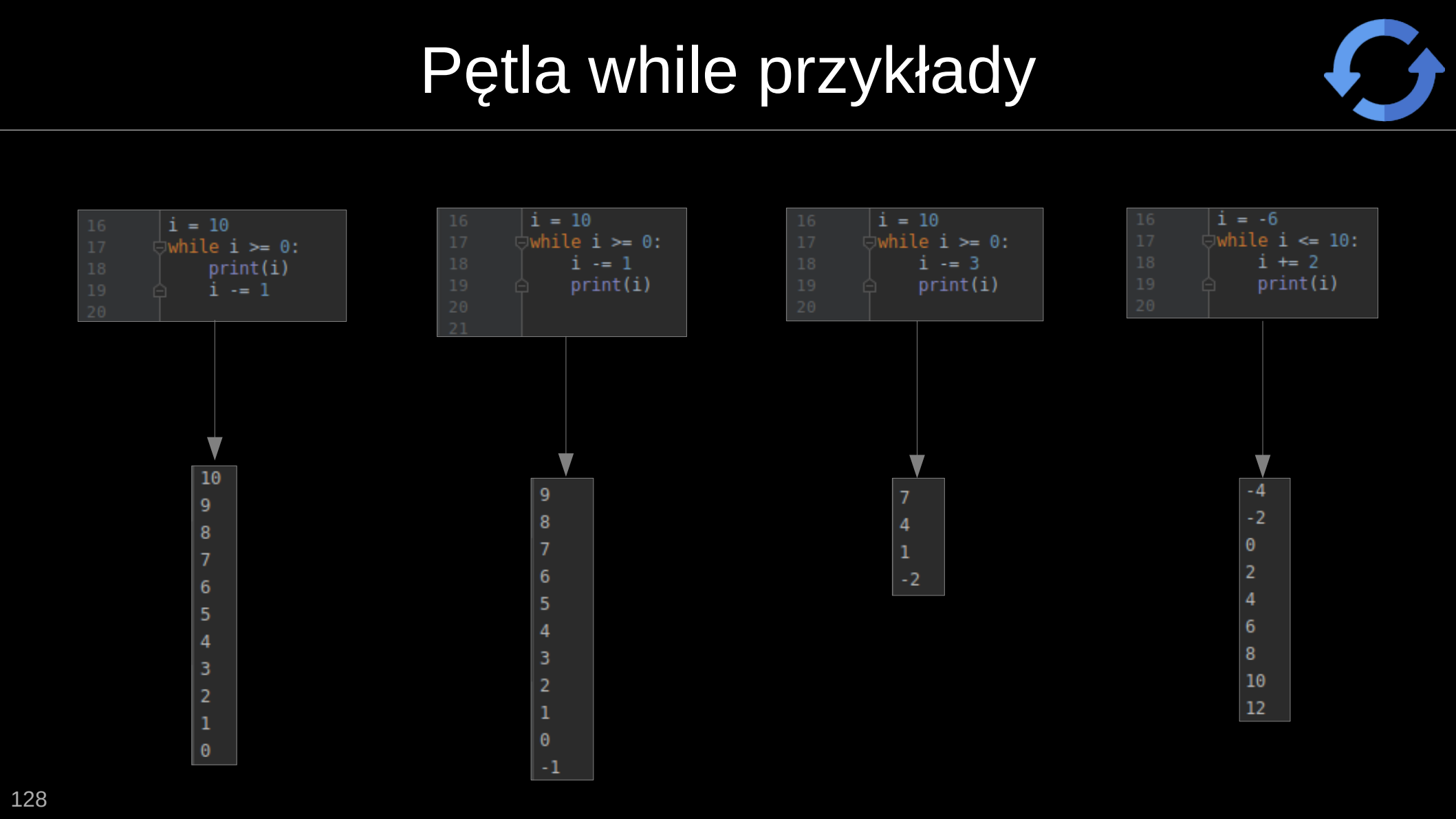
Pętla nieskończona, ponieważ w warunku wprowadzone jest True (prawda). Warunek będzie zawsze spełniony, więc wykona się zawartość pętli.

```
counter = 5
while counter != 0:
    print(counter)
```

```
5
5
5
5
5
Process finished with exit code -1
```

Pętla nieskończona, ponieważ w warunku zawsze wyjdzie prawda, ponieważ liczba 5 jest zawsze różna od 0. Warunek będzie zawsze spełniony, więc wykona się zawartość pętli.

Pętla while przykłady



```
16      i = 10
17      while i >= 0:
18          print(i)
19          i -= 1
20
```

```
10
9
8
7
6
5
4
3
2
1
0
```

```
16      i = 10
17      while i >= 0:
18          i -= 1
19          print(i)
20
21
```

```
9
8
7
6
5
4
3
2
1
0
-1
```

```
16      i = 10
17      while i >= 0:
18          i -= 3
19          print(i)
20
```

```
7
4
1
-2
```

```
16      i = -6
17      while i <= 10:
18          i += 2
19          print(i)
20
```

```
-4
-2
0
2
4
6
8
10
12
```

Ćwiczenie

Eksperymenty z pętlą while

ćwiczenie 15 minutowe

- Napisz program który zapyta użytkownika o liczbę a następnie policzy ile jest liczb parzystych większych od 0, nie większych od tej liczby oraz je wypisze.

Pętla for



Instrukcja for wykonuje pewien fragment kodu określoną (z góry znaną) ilość razy, przeznaczona jest głównie do iterowania po danych. w pętli for nie ma warunku logicznego kończącego pętlę. Pętla kończy swoje działanie gdy wykona operacje na całym zbiorze na którym została uruchomiona.

Funkcja ***range()*** zwraca ciąg liczb od 0 do wskazanej liczby (bez niej).

pass jest specjalną pustą instrukcją - nie robi nic.

```
for i in range(10):
    pass
```

Pętla for



Zmienna do obsługi pętli

Definicja z jakiego przedziału liczby mają zostać wypisane

```
for liczba in range(0, 10):
    print(liczba)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
0
1
2
3
4
5
6
7
8
9
```

Stan zmiennej liczba w każdym obiegu pętli:

liczba = 0
liczba = 1
liczba = 2
liczba = 3
liczba = 4
liczba = 5
liczba = 6
liczba = 7
liczba = 8
liczba = 9

Pętla for



```
for i in range(10):
    print(i)
```

Wynik:

```
0
1
2
3
4
5
6
7
8
9
```

Iteracja 1: pod i zapisze się wartość 0 wypisze 0 na ekranie.
Iteracja 2: pod i zapisze się wartość 1 wypisze 1 na ekranie.
Iteracja 3: pod i zapisze się wartość 2 wypisze 2 na ekranie.
Iteracja 4: pod i zapisze się wartość 3 wypisze 3 na ekranie.
Iteracja 5: pod i zapisze się wartość 4 wypisze 4 na ekranie.
Iteracja 6: pod i zapisze się wartość 5 wypisze 5 na ekranie.
Iteracja 7: pod i zapisze się wartość 6 wypisze 6 na ekranie.
Iteracja 8: pod i zapisze się wartość 7 wypisze 7 na ekranie.
Iteracja 9: pod i zapisze się wartość 8 wypisze 8 na ekranie.
Iteracja 10: pod i zapisze się wartość 9 wypisze 9 na ekranie.

Pętla for iteracja po łańcuchu znaków



```
for c in "Ala ma kota":
    print(c)
```

Wynik:

```
A
l
a

m
a

k
o
t
a
```

```
for c in "super tekst":
    if c != " ":
        print(c, end="-")
```

Wynik:

```
s-u-p-e-r-t-e-k-s-t-
```

Pętla for iteracja po łańcuchu znaków



```
for c in "Ala ma kota":
    print(c, end="")
```

Wynik:

```
Ala ma kota
```

```
for c in "Ala ma kota":
    print(c + "*", end="")
```

Wynik:

```
A*l*a* *m*a* *k*o*t*a*
```

Ciało pętli for



Przestrzeń ogólna

Ciało pętli for

Przestrzeń ogólna

```
for liczba in range(0, 10):
    print(liczba)

print(liczba)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
0
1
2
3
4
5
6
7
8
9
9
```

Ciało pętli for



Przestrzeń ogólna

Ciało pętli for

Ciało instrukcji warunkowej if

Przestrzeń ogólna

```
for liczba in range(0, 10):
    if liczba % 2 == 0:
        print(liczba)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
0
2
4
6
8
```

Czy 0 % 2 == 0
Czy 1 % 2 == 0 Reszta z dzielenia to 1
Czy 2 % 2 == 0
Czy 3 % 2 == 0 Reszta z dzielenia to 1
Czy 4 % 2 == 0
Czy 5 % 2 == 0 Reszta z dzielenia to 1
Czy 6 % 2 == 0
Czy 7 % 2 == 0 Reszta z dzielenia to 1
Czy 8 % 2 == 0
Czy 9 % 2 == 0 Reszta z dzielenia to 1

Ćwiczenie

Eksperymenty z pętlą for

ćwiczenie 15 minutowe

- Poeksperymentuj z pętlą for zmodyfikuj program tak aby zamiast pętli while używał pętli for, użyj funkcji range() poeksperymentuj z pętlą for tak aby iterować po ciągu znaków poeksperymentuj z funkcją range() i jej odmianami.

Instrukcje break

Instrukcja break jest przydatną instrukcją języka Python i umożliwia przerwanie działania pętli w dowolnym momencie.

```
counter = 0
while True:
    print("test!")
    counter += 1
    if counter == 5:
        break
```

```
test!
test!
test!
test!
test!
```

1 iteracja: warunek spełniony ponieważ True, wypisz „test!” zwiększ wartość counter o 1. counter == 1 Sprawdź (if) czy counter jest równe 5. Nie jest równe 5, więc nie wykonuje break.
2 iteracja: warunek spełniony ponieważ True, wypisz „test!” zwiększ wartość counter o 1. counter == 2 Sprawdź (if) czy counter jest równe 5. Nie jest równe 5, więc nie wykonuje break.
3 iteracja: warunek spełniony ponieważ True, wypisz „test!” zwiększ wartość counter o 1. counter == 3 Sprawdź (if) czy counter jest równe 5. Nie jest równe 5, więc nie wykonuje break.
4 iteracja: warunek spełniony ponieważ True, wypisz „test!” zwiększ wartość counter o 1. counter == 4 Sprawdź (if) czy counter jest równe 5. Nie jest równe 5, więc nie wykonuje break.
5 iteracja: warunek spełniony ponieważ True, wypisz „test!” zwiększ wartość counter o 1. counter == 5 **Sprawdź (if) czy counter jest równe 5. Jest równe 5, więc wykonuje break.**

Instrukcje continue

Instrukcja continue jest instrukcją języka Python i umożliwia pominięcie dalszego wykonania pętli i natychmiastowe przejście do następnej iteracji.

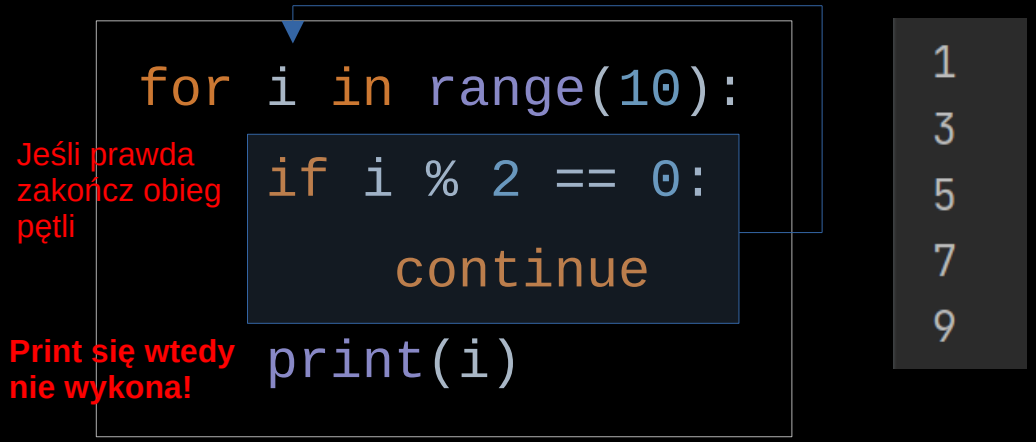
```
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
5
7
9
```

Range() mówi, że sprawdzamy liczby od 0 do 9 i będziemy je podstawiać pod zmienną i. Jeśli i modulo 2 jest równe 0, to oznacza, że liczba jest parzysta i nie wypisze się ponieważ kończona jest obecna iteracja pętli.

Instrukcje continue

Instrukcja continue jest instrukcją języka Python i umożliwia pominięcie dalszego wykonania pętli i natychmiastowe przejście do następnej iteracji.



Range() mówi, że sprawdzamy liczby od 0 do 9 i będziemy je podstawiać pod zmienną i. Jeśli i modulo 2 jest równe 0, to oznacza, że liczba jest parzysta i nie wypisze się ponieważ kończona jest obecna iteracja pętli.

Ćwiczenie

Eksperymenty z pętlami

ćwiczenie 15 minutowe

- Poeksperymentuj z pętlami i napisz program który będzie pytał użytkownika o podanie liczby tak długo aż użytkownik poda poprawną liczbę

w celu ułatwienia rozgrywki możesz dodać komunikaty pomagające odgadnąć liczbę.

Ćwiczenie

Eksperymenty z pętlami

ćwiczenie 15 minutowe

- Poeksperymentuj z pętlami zmodyfikuj kalkulator tak aby obliczał wartości dla podanych przez użytkownika dwóch liczb i wybranej operacji oraz kończył działanie dopiero na życzenie użytkownika.

Pętle i else

Używając pętli, możemy tak samo jak w przypadku instrukcji warunkowej if skorzystać z instrukcji else. Instrukcję else zapisujemy po ciele pętli (zarówno while jak i for) i wykona się ona zawsze raz, bez względu na to czy wykonany zostanie kod w ciele pętli.

```
counter = 0
while counter < 5:
    print(counter)
    counter += 1
else:
    print("else", counter)
```

```
0
1
2
3
4
else 5
```

Pętle i else

Używając pętli, możemy tak samo jak w przypadku instrukcji warunkowej if skorzystać z instrukcji else. Instrukcję else zapisujemy po ciele pętli (zarówno while jak i for) i wykona się ona zawsze raz, bez względu na to czy wykonany zostanie kod w ciele pętli.

```
i = 1
for i in range(2, 1):
    print(i)
else:
    print("else", i)
```

```
else 1
```

Warunki - przykłady

```
a = 5
b = 10
c = 15

if a == 5:
    print("a to liczba 5")

elif b == 10:
    print("b to liczba 10")

elif c == 15:
    print("c to liczba 15")

else:
    print("Nic z powyższych")
```

Jeśli a jest równe 5 to:
Wypisz tekst: `a to liczba 5`

```
a to liczba 5
```

W przeciwnym razie Jeśli b jest równe 10 to:
Wypisz tekst: `b to liczba 10`

W przeciwnym razie Jeśli b jest równe 10 to:
Wypisz tekst: `b to liczba 10`

W przeciwnym w przeciwnym wypadku to:
Wypisz tekst: `b Nic z powyższych`

Warunki - przykłady

```
16  a = 5
17  b = 10
18  c = 15
19
20  if a == 5:
21      print("a to liczba 5")
22
23  elif b == 10:
24      print("b to liczba 10")
25
26  elif c == 15:
27      print("c to liczba 15")
28
29  else:
30      print("Nic z powyższych")
31
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
a to liczba 5
```

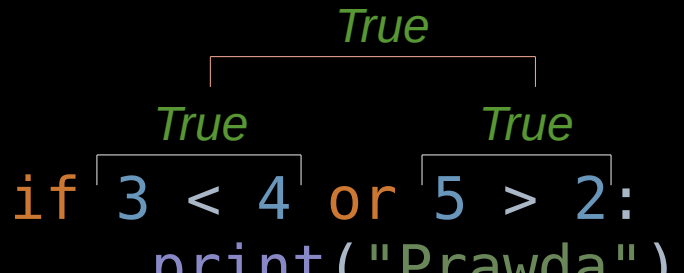
Logiczne or (lub) - przykłady

```
print(True or True)
print(False or True)
print(1 == 1 or 2 == 1)
print("Jakis teskst" == "tekst 2 ")

print("Tekst" == 1)
print("Test" == "Test")
print(10 == 1 or 1000 == 1000)
print(10 < 1 or 1000 > 1000)
```

```
True
True
True
False

False
True
True
False
```



True or True zwraca True
True or False zwraca True
False or True zwraca True
False or False zwraca False

Logiczne and (i) - przykłady

```
print(True and True)
print(False and True)
print(1 == 1 and 2 == 1)
print("Jakis teskst" == "tekst 2 ")
```

```
True
False
False
False
```

```
print("Tekst" == 1)
print("Test" == "Test")
print(10 == 1 and 1000 == 1000)
print(10 < 1 and 1000 > 1000)
```

```
False
True
False
False
```

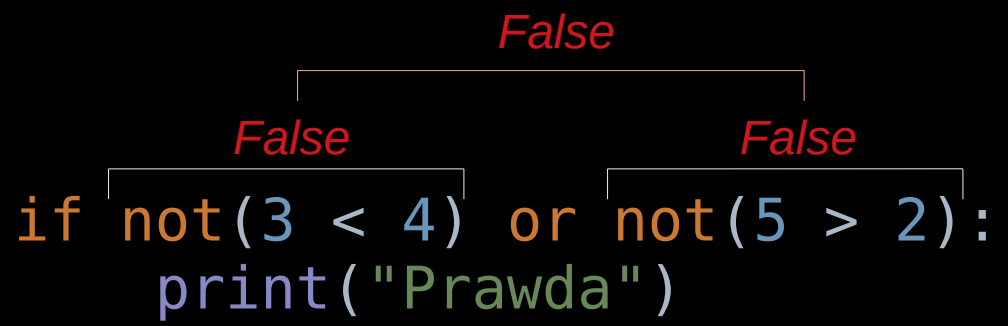
```
        False
   ┌──────────────┐
  True           False
┌──┴──┐       ┌──┴──┐
if 3 < 4 and 1 > 2:
    print("Prawda")
```

True and True zwraca True
True and False zwraca False
False and True zwraca False
False and False zwraca False

Logiczne not (nie) - przykłady

```
print(not True and True)                    False
print(not(False and True))                  True
print(1 == 1 and 2 == 1)                    False
print("Jakis teskst" == "tekst 2 ")         False

print(not("Tekst" == 1))                    True
print(not "Test" == "Test")                 False
print(not(10 == 1) and 1000 == 1000)        True
print(not 10 < 1 and 1000 > 1000)           False
```



not True zwraca False
not False zwraca True

Operatory bitowe

Istnieją cztery operatory, które pozwalają manipulować pojedynczymi bitami zawierającymi dane. Nazywamy je operatorami bitowymi.

& - koniunkcja bitowa

| - alternatywa bitowa

~ - negacja bitowa

^ - alternatywa rozłączna bitowa (xor)

A	B	A & B	A | B	A ^ B
1	1	1	1	0
1	0	0	1	1
0	0	0	0	0
0	1	0	1	1

```
a = 14 #1110
b = 10 #1010
print(a & b) # 1010 # 10
print(a | b) # 1110 # 14
print(a ^ b) # 0100 # 4
```

```
10
14
4
```

Binarne przesunięcie

Python oferuje operator binarnego przesunięcia w lewo i w prawo

<< przesunięcie bitowe w lewo

>> przesunięcie bitowe w prawo

wartosc << bity
wartosc >> bity

```
print(8 << 1) # 16
print(8 >> 2) # 2
```

```
16
2
```

Ćwiczenie

Eksperymenty z operatorami bitowymi

ćwiczenie 5 minutowe

- Wykorzystując operatory bitowe używając funkcji print przećwicz ich działanie.

Algorytmy

Algorytm to skończony ciąg jasno zdefiniowanych czynności koniecznych do wykonania pewnego rodzaju zadań, sposób postępowania prowadzący do rozwiązania problemu. Można go przedstawić na schemacie blokowym.

Słowo „algorytm” pochodzi od łacińskiego słowa algorithmus, oznaczającego wykonywanie działań przy pomocy liczb arabskich (w odróżnieniu od abacism – przy pomocy abakusa), które z kolei wzięło się od nazwy „Algoritmi”, zlatynizowanej wersji nazwiska „al-Chwarizmi” Abu Abdullaha Muhammada ibn Musy al-Chuwarizmiego, matematyka perskiego z IX wieku.

Zadaniem algorytmu jest przeprowadzenie systemu z pewnego stanu początkowego do pożądanego stanu końcowego. Badaniem algorytmów zajmuje się algorytmika. Algorytm może zostać zaimplementowany w postaci programu komputerowego.

Jako przykład stosowanego w życiu codziennym algorytmu podaje się często przepis kulinarny. Dla przykładu, aby ugotować bigos, należy w określonej kolejności oraz odstępach czasowych (imperatyw czasowy) dodawać właściwe rodzaje kapusty i innych składników. Może istnieć kilka różnych przepisów dających na końcu bardzo podobną potrawę. Przykład ten ma wyłącznie charakter poglądowy, ponieważ język przepisów kulinarnych nie został jasno zdefiniowany.

Reprezentacja algorytmów

Algorytm opisujący operacje do wykonania może zostać zapisany w różny sposób. Może to być:

- zapis słowny
- lista kroków
- pseudokod
- drzewo algorytmu
- schemat blokowy

Algorytmy - przykład

Opis słowny

W opisie słownym operacje, które należy wykonać, zapisywane są za pomocą zwykłego tekstu. Ten sposób zapisu algorytmu stosowany jest we wstępnej fazie opisu problemu, gdy trzeba w sposób ogólny przedstawić operacje do wykonania, bez określania szczegółów dotyczących rozwiązania problemu.

Lista kroków

Na liście kroków każda operacja, którą należy wykonać, jest zapisywana w postaci numerowanego kroku. Lista kroków pozwała dokładnie zdefiniować cały algorytm.

Pseudokod

Pseudokod to opis słowny przypominający zapis kroków algorytmu, który może zawierać instrukcje z języka programowania. Tworząc pseudokod, najczęściej używamy słów języka naturalnego do określenia kroków algorytmu (np. wczytaj dane, oblicz wartość), ale mogą pojawić się w nim elementy zawierające symboliczny opis działań (np. a=5, powtarzaj ... aż do ...).

Algorytmy - przykład

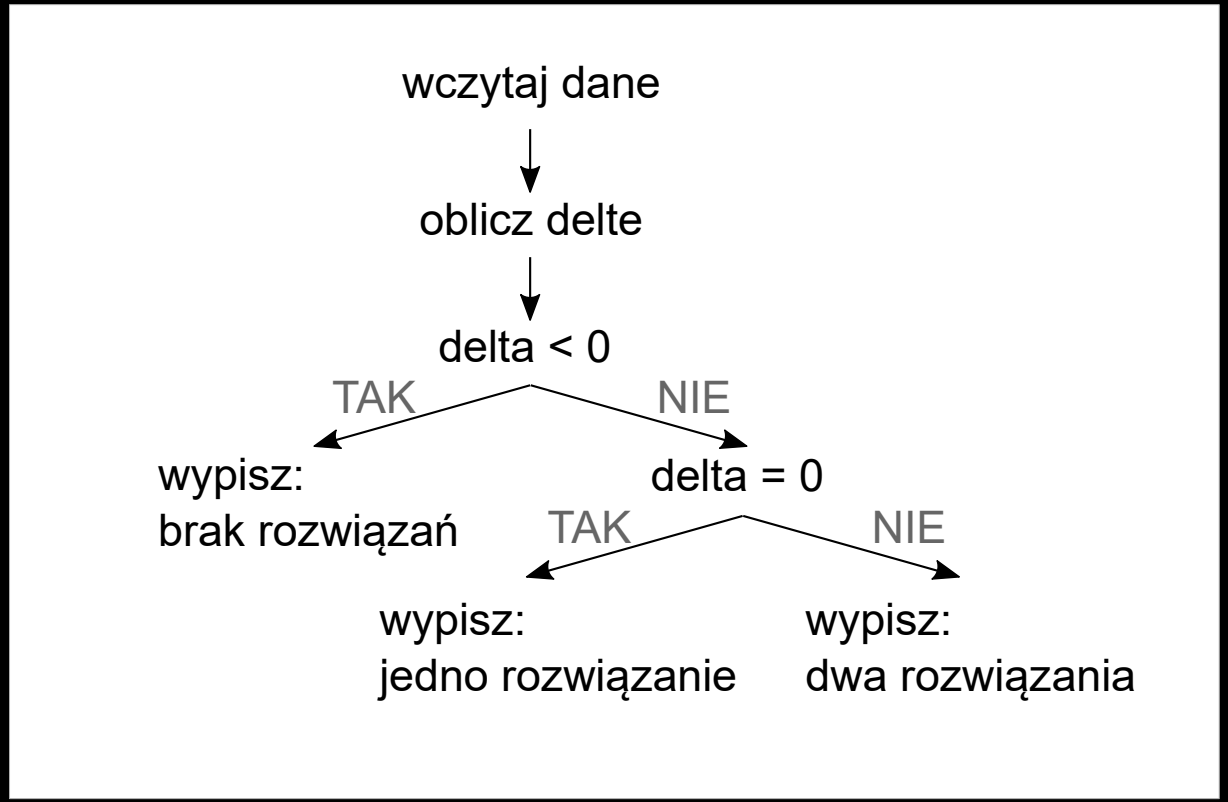
Drzewo algorytmu

Drzewo algorytmu to reprezentacja graficzna algorytmu. W schemacie drzewa wyróżnione są: jeden główny element, tj. korzeń (wierzchołek), który stanowi początek algorytmu, gałęzie (wierzchołki pośrednie), które są reprezentacją wykonywanych operacji, oraz liście (wierzchołki końcowe), które reprezentują otrzymane wyniki. Drzewo algorytmu może zostać przedstawione jako graf.

Schemat blokowy

W schemacie blokowym operacje, które należy wykonać, są przedstawione w postaci graficznej z użyciem symboli.

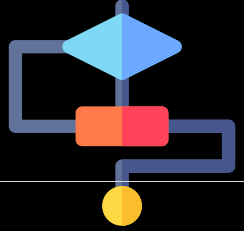
Drzewo algorytmu - przykład



Algorytmy - przykład

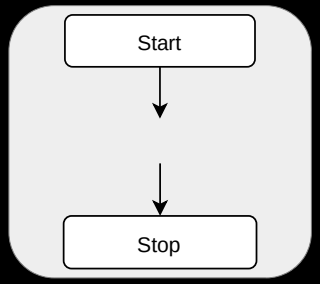
Algorytm przyniesienia szklanki Coca-Coli z kuchni zapisany jako lista kroków:

1) Idź do kuchni.
2) Weź szklankę.
3) Weź i odkręć butelkę Coca-Coli.
4) Podstaw szklankę i nalej Coca-Colę.
5) Zakręć butelkę.
6) Zanieś szklankę osobie, którą prosiła o Coca-Colę.

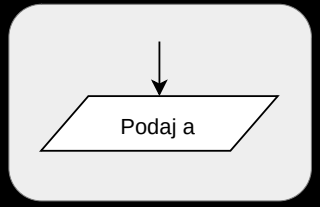


Schematy blokowe

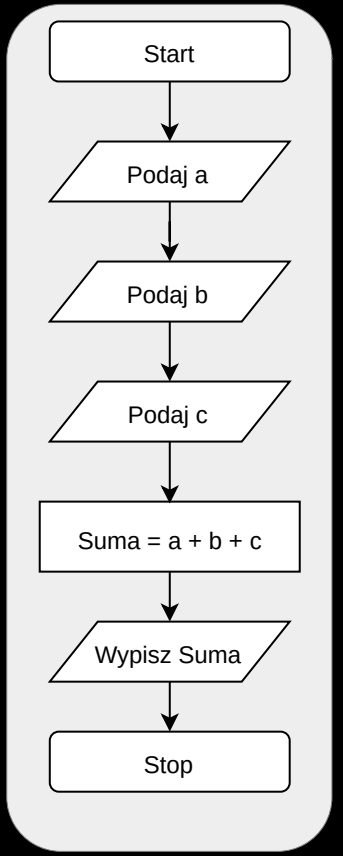
Schemat blokowy – narzędzie służące do przedstawienia kolejnych czynności w projektowanym algorytmie.



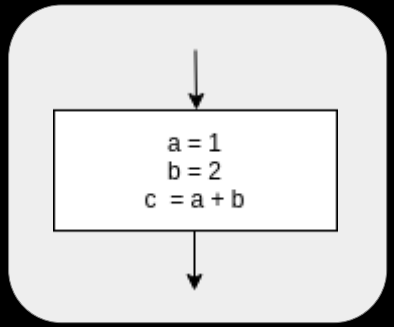
Początek lub koniec algorytmu, w jednym algorytmie są dwa takie elementy (Start, Stop).
Start posiada tylko jedno połączenie wychodzące.
Stop posiada tylko jedno połączenie wchodzące.



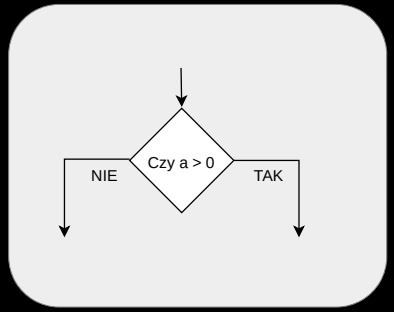
Blok wejścia/wyjścia (input/output) np. wypisuje wartość zwróconą przez algorytm lub pobiera dane od użytkownika.
Posiada jedno połączenie wchodzące i jedno wychodzące.



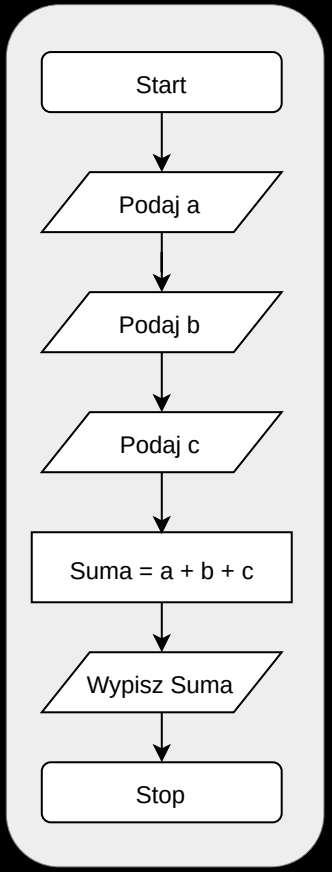
Schematy blokowe



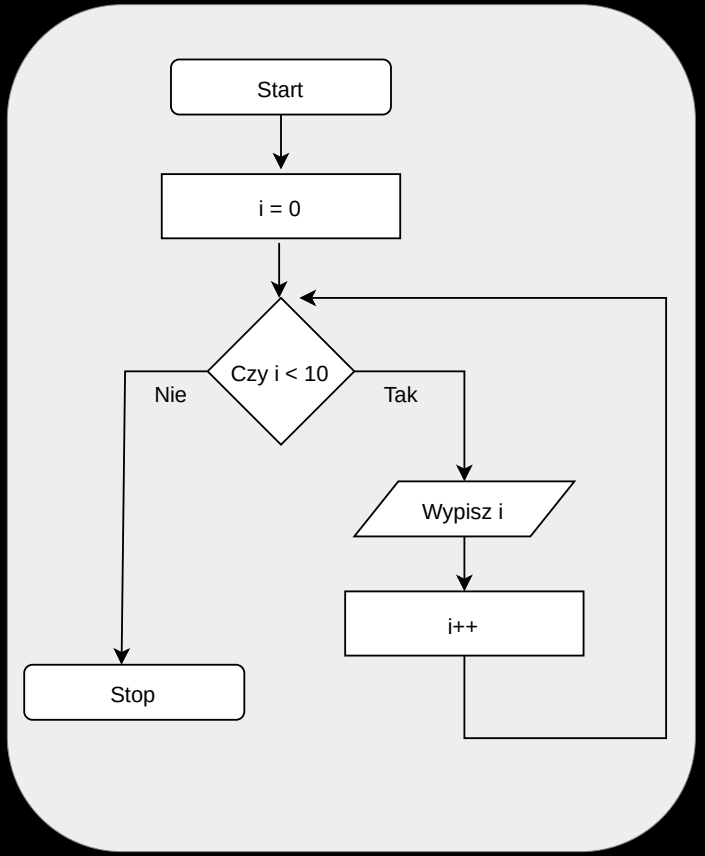
Wykonywanie działań (blok operacyjny) blok w którym wykonuje się różnego typu operacje np. obliczenia. Posiada jedno połączenie wchodzące i jedno wychodzące. W jednym algorytmie może być kilka takich bloków i jeden blok może przyjmować więcej niż jedno wyrażenie.



Sprawdzanie warunku (blok warunkowy albo decyzyjny). Wchodzi do niego jedno połączenie, a wychodzą dwa (TAK i NIE).
W jednym schemacie może być wiele takich bloków.



Odwzorowanie pętli za pomocą schematu blokowego



Pętla WHILE

```
i = 0
while i < 10:
    print(i)
    i+=1
```

Pętla zwróci taki wynik:

```
0
1
2
3
4
5
6
7
8
9
```

Ćwiczenie w grupie

Tworzenie algorytmu

ćwiczenie 30 minutowe

- Proszę dobrać się w grupy oraz wymyślić jakiś algorytm, który zostanie opisany za pomocą:
  - schematu blokowego,
  - listy kroków,
  - drzewa algorytmu.
- Na podstawie opracowanych reprezentacji algorytmu, proszę napisać kod.

Należy skorzystać z narzędzia draw.io (https://app.diagrams.net)

Listy

Listy służą do przechowywania zbioru danych. **[2, 56, 56.5]**

Listy mogą przechowywać dowolną ilość elementów. 

Elementy listy mogą być dowolnego typu. **2** **6.5** **"tekst"**

Listy w języku Python zapisujemy za pomocą nawiasów kwadratowych. **[ ]**

```
lista = []
lista2= [1, 2, 3, 4, 5]
lista3 = [1, "a", True]

print(lista2)
print(lista2[2])
print(lista2[4])
```

```
[1, 2, 3, 4, 5]
3
5
```

Listy - indeksowanie

Do modyfikowania elementów listy posługujemy się indeksami elementulisty.

Listy indeksowane są od 0.

Modyfikacja pierwszego elementu listy:

```
Lista = [1, 2, 3, 6]
```

Indeks: 0 1 2 3

```
lista = [0, 1, 2]
lista[0] = 1
print(lista) # [1, 1, 2]
lista[0] = lista[2]
print(lista) # [2, 1, 2]
```

```
[1, 1, 2]
[2, 1, 2]
```

```
Lista = [1, 2, 3, 6]

print(lista[0]) #1
print(lista[2]) #3
```

```
1
3
```

Listy - sprawdzanie długości

Lista jest dynamicznym elementem języka Python i może być dowolnie modyfikowana w trakcie działania programu.

Aby sprawdzić aktualną długość listy należy użyć funkcji **len()**

```
lista = [1, 2, 3]
print(len(lista)) # 3

lista2 = [1, 2, 3, "tekst", True]
print(len(lista2)) # 5
```

```
3
5
```

Lista - usuwanie elementów

W dowolnym momencie działania programu możemy usunąć elementy z listy, służy do tego instrukcja **del**

```
del lista[1]
```

Usunięcie elementu listy modyfikuje długość listy oraz modyfikuje indeksy elementów występujących po elemencie usuniętym.

```
lista = [1, 2, 3, 6]
del lista[1]
print(len(lista)) # 3
print(lista[1]) # 3
```

```
3
3
```

Listy - indeksowanie

Jest zbiorem różnych wartości, dane zapisane w liście mogą być różnego typu. Lista jest typem, którym można w łatwy sposób manipulować (dodawanie, uduwanie, edytowanie jej wartości). Indeksy w liście rozpoczynają się od 0.

Wartości (elementy) listy

```
moja_lista = [1, 3, 4, 3, 1, 8]
```

Edycja wartości listy:

```
moja_lista = [1, 3, True, 3, "Ania", 8.4]
```

Dodanie elementu do listy:

```
moja_lista.append(5)
```

Sprawdzanie długości listy:

```
len(moja_lista)          #zwróci długość listy, czyli wartość 6
```

Usunięcie elementu listy:

```
del(moja_lista[0])
```

Indeksy listy:

Indeksy listy rozpoczynają się od 0, można się do nich odwoływać w poniższy sposób:

```
moja_lista = [1, 3, 4, 3, 1, 8]
```

Indeks 0 Indeks 1 Indeks 2 ltd...

```
moja_lista = [0]      #Zwróci wartość 1
moja_lista = [1]      #Zwróci wartość 3
```

Lista - ujemne indeksy

Dostęp do elementów listy możemy również uzyskać za pomocą indeksów ujemnych. Python zezwala na indeksy ujemne w celu uzyskanie dostępu do elementów listy "od końca".

```
lista = [1, 2, 3, 4]
print(lista[len(lista) - 1]) #4
print(lista[-1]) #4
```

```
4
4
```

Metody w Python

Metody to specyficzny rodzaj funkcji.

Funkcja nie operuje na żadnym zbiorze danych, tylko na argumentach przekazanych do niej.

Metoda operuje na zbiorze danych na jakim została wywołana.

```
wynik = funkcja(argument)
wynik = dane.metoda(argument)
```

Lista - dodawanie elementów

W celu dodania elementów do listy która istnieje możemy użyć jednej z dwóch metod append() i insert().

append(wartosc) - dodaje nowy element wartosc na końcu listy

insert(miejsce, wartosc) - dodaje nowy element wartosc do listy na miejscu wskazanym przez miejsce.

```
lista = []
lista.append(3)
print(lista)
lista.insert(0, 90)
print(lista)
```

```
[3]
[90, 3]
```

Lista - dodawanie elementów

```
lista = [55, 66, 77]
lista.append(44)
print(lista)
lista.insert(0, 99)
print(lista)
```

```
[55, 66, 77, 44]
[99, 55, 66, 77, 44]
```

Lista - przykłady

```
moja_lista = [1, 3, True, 3, "Ania", 8.4]
print(len(moja_lista))
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
6
```

```
moja_lista = [1, 3, True, 3, "Ania", 8.4]
del(moja_lista[0])
print(moja_lista)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
[3, True, 3, 'Ania', 8.4]
```

```
moja_lista = [1, 3, True, 3, "Ania", 8.4]
moja_lista.append(45)
print(moja_lista)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
[1, 3, True, 3, 'Ania', 8.4, 45]
```

```
moja_lista = [1, 3, True, 3, "Ania", 8.4]
print(moja_lista[4])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
Ania
```

```
moja_lista = [1, 3, True, 3, "Ania", 8.4]
moja_lista[4] = "Adam"
print(moja_lista[4], )
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
Adam
```

Lista - przykłady

```
moja_lista = [1, 3, 4, 3, 25, 8.4, 56]

print(moja_lista[4:6])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
[25, 8.4]
```

```
moja_lista = [1, 3, 4, 3, 25, 8.4, 56]

print(moja_lista[2:])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
[4, 3, 25, 8.4, 56]
```

```
moja_lista = [1, 3, 4, 3, 25, 8.4, 56]

print(75 not in moja_lista)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
True
```

Ćwiczenie

Eksperymenty z listami

Ćwiczenie 10 minutowe

- Wykorzystując poznane elementy języka Python do przetwarzania list sprawdź jak zachowują się one w praktyce utwórz listę, wyświetl całą listę, pojedynczy element listy, modyfikuj elementy listy, usuń istniejące elementy i dodawaj nowe

Pętla for - iteracja po zbiorze danych

Pętla for przeznaczona jest do iterowania po zbiorach danych. Możemy wykorzystać ją do iterowania elementów list.

```
Lista = [1, 2, 3, 4, 5, 6]
for i in lista:
    print(i)
```

```
1
2
3
4
5
6
```

Pętla for - iteracja po zbiorze danych

```python
moja_lista = [1, 3, True, 3, "Ania", 8.4]


for i in moja_lista:
    print(i)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
1
3
True
3
Ania
8.4
```

```python
moja_lista = [1, 3, 4, 3, 25, 8.4, 56]


moja_lista.sort()

for i in moja_lista:
    print(i)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
1
3
3
4
8.4
25
56
```

```python
moja_lista = [i for i in range(9)]


print(moja_lista)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Metoda sort() list

Metoda sort() sortuje elementy listy.

```
lista = [1, 2, 3, 34, 5, 34, 0]
lista.sort()
print(lista)
```

```
[0, 1, 2, 3, 5, 34, 34]
```

Metoda sort() list

Metoda sort() sortuje elementy listy.

```
lista = [4, 2, 5, 8]
lista.sort()
print(lista) # [2, 4, 5, 8]
lista.sort(reverse=True)
print(lista) # [8, 5, 4, 2]
```

```
[2, 4, 5, 8]
[8, 5, 4, 2]
```

Ćwiczenie

Eksperymenty z listami

Ćwiczenie 45 minutowe

- Wykorzystując poznane elementy języka Python do przetwarzania list i pętli, zmodyfikuj kalkulator w taki sposób, aby dał możliwość wprowadzenia więcej niż 2 liczb oraz przeprowadził na wprowadzonym zbiorze wybraną operację.
- Posortuj przykładowe zbiory danych zapisane w liście wykorzystując metodę sort()

Modyfikacja elementów list

```
lista = [1, 2, 3]
lista[0], lista[2] = lista[2], lista[0]
print(lista) # [3, 2, 1]
```

```
[3, 2, 1]
```

```
lista = [1, 2, 3]
lista.reverse()
print(lista) # [3, 2, 1]
```

Ćwiczenie

Eksperymenty z listami i pętlami

Ćwiczenie 35 minutowe

- Wykorzystując poznane elementy języka Python do przetwarzania list i pętli, zapoznaj się z algorytmem sortowanie bąbelkowego i zaimplementuj jego prostą wersję

https://www.youtube.com/watch?v=Iv3vgjM8Pv4

Referencja a wartość

Zmienna w Pythonie może przyjmować wartość (literał) albo może wskazywać na pewne miejsce w pamięci gdzie znajduje się bardziej złożona dana (np. lista).

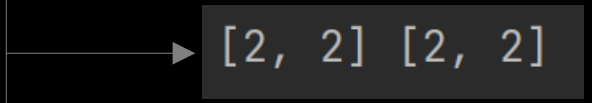
Różnica pomiędzy wartością a referencją jest znacząca. Przypisanie wartości zmiennej przechowującej literał tworzy kopię tej wartości i przypisuje do nowej zmiennej. Przypisanie wartości zmiennej wskazującej na „nie literał” tak naprawdę utworzy nową referencje (wskaźnik) na miejsce w pamięci gdzie znajdują się dane.

Referencja a wartość

```
x = 1
y = x
y = 2
print(x, y) # 1 2
```




```
x = [1, 2]
y = x
y[0] = 2
print(x, y) # [2, 2] [2, 2]
```



Listy - wycinki

Wycinki to tworzenie nowej listy na podstawie istniejącej listy. Aby utworzyć nową listę na podstawie już istniejącej używamy operatora [X:Y] gdzie X to element od którego rozpoczynamy kopiowanie listy a Y to element do którego kopiujemy elementy (bez tego elementu).

```
lista = [1, 2, 3, 4, 5]
lista2 = lista[0:2]
lista3 = lista[4:6]
print(lista2, lista3) # [1, 2] [5]
```

```
[1, 2] [5]
```

Listy - wycinki

Za pomocą wycinków możemy kopiować całe listy

```
lista = [1, 2, 3, 4, 5]
lista2 = lista[:]
print(lista2) # [1, 2, 3, 4, 5]
```

```
[1, 2, 3, 4, 5]
```

możemy również użyć metody copy() - lista.copy()
Dozwolone są również indeksy ujemne:

```
lista = [1, 2, 3, 4, 5]
lista2 = lista[-2:-1]
print(lista2) # [4]
```

```
[4]
```

Listy - wycinki

Za pomocą wycinków możemy usuwać elementy listy:

```
lista = [0, 1, 2, 3, 4, 5]
del lista[1:3]
print(lista) # [0, 3, 4, 5]
```

```
[0, 3, 4, 5]
```

Jak i całe zawartości listy:

```
lista = [1, 2, 3, 4, 5]
del lista[:]
print(lista) # []
```

```
[]
```

Ćwiczenie

Eksperymenty z listami

Ćwiczenie 15 minutowe

- Wykorzystując poznane elementy języka Python do przetwarzania list sprawdź jak zachowuje się referencja utwórz nowe tablice za pomocą wycinków, operuj na nich i sprawdzaj wyniki za pomocą funkcji print.

Listy - wycinki

Operator in w połączeniu z listami daje nam informację czy wskazany przez nas element znajduje się w liście.

Alternatywnie operator not in sprawdza czy element nie znajduje się w liście.

```
lista = [1, 2, 3, 4, 5]
print(1 in lista) # True
print(3 not in lista) # False
```

```
True
False
```

Ćwiczenie

Eksperymenty z listami

Ćwiczenie 10 minutowe

- symulacja totolotka napisz prosty program który poprosi użytkownika o podane 6 liczb z zakresu 1-49 i wyświetli ile wskazanych przez użytkownika liczb pokrywa się ze zdefiniowaną w programie listą 6 liczb z tego przedziału.

Do wykonania należy użyć metodę randint()

```
import random
print(random.randint(3, 6))
print(random.randint(3, 6))
```

```
3
5
```

Wyrażenia listowe

Python umożliwia tworzenie list wraz z wypełnieniem ich pożądanymi wartościami.

Możemy np. stworzyć listę której kolejne elementy będą wielokrotnościami liczby trzy.

```
lista = [x * 2 for x in range(5)]
print(lista) # [0, 2, 4, 6, 8]
```

```
[0, 2, 4, 6, 8]
```

Listy wielowymiarowe

Python umożliwia również tworzenie list wielowymiarowych.

Listy wielowymiarowe to nic innego jak listy umieszczone w listach.

```
lista = []
for i in range(3):
    lista.append([])
    for j in range(3):
        lista[i].append(j)
print(lista) # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
[[0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
lista = [[j for j in range(3)] for i in range(3)]
print(lista) # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
[[0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

Ćwiczenie

Eksperymenty z listami i pętlami

ćwiczenie 45 minutowe

- Napisz program z wykorzystaniem pętli i list który wypisze rezultat tabliczki mnożenia (wynik przemnożenia przez siebie wartości w liscie):

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[2, 4, 6, 8, 10, 12, 14, 16, 18]
[3, 6, 9, 12, 15, 18, 21, 24, 27]
[4, 8, 12, 16, 20, 24, 28, 32, 36]
[5, 10, 15, 20, 25, 30, 35, 40, 45]
[6, 12, 18, 24, 30, 36, 42, 48, 54]
[7, 14, 21, 28, 35, 42, 49, 56, 63]
[8, 16, 24, 32, 40, 48, 56, 64, 72]
[9, 18, 27, 36, 45, 54, 63, 72, 81]
```

Ćwiczenie

Projekt

ćwiczenie 60 minutowe

- napisz prosty kalkulator wagi który będzie przechowywał dane w tablicy dwuwymiarowej miesiące i dni, będzie prosił użytkownika o wprowadzenie wagi w konkretnym dniu miesiąca a po zakończeniu wprowadzania danych policzy i wypisze kilka informacji o wadze (średnia waga, najniższa, itp.)

Ćwiczenie

Projekt z wykorzystaniem list

ćwiczenie 40 minutowe

Symulacja Totolotka

- Napisz prosty program który poprosi użytkownika o podane 6 liczb z zakresu 1-49 i wyświetli ile wskazanych przez użytkownika liczb pokrywa się ze zdefiniowaną w programie listą 6 liczb z tego przedziału.